

JEPA

📅 date	2026년 10월 1일
# Num	18
🔗 발제자료	https://canva.link/c7le9vvlsxzvykt
≡ 유형	BOAZ 학기 5주차 (1) - 발제

V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning

Background

[Self-Supervised Learning\(SSL\) 세 가지 패러다임](#)

[ViT, 패치, Tubelet](#)

[Representation Collapse와 EMA / Stop-gradient](#)

[Frozen Evaluation과 Attentive Probe](#)

[World Model과 Model Predictive Control\(MPC\)](#)

[3D-RoPE](#)

1. Introduction

2. V-JEPA 2: Scaling Self-Supervised Video Pretraining

[2.1 Methodology](#)

[2.2 Scaling Self-Supervised Video Learning](#)

[2.3 Pretraining Dataset](#)

[2.4 Pretraining Recipe](#)

3. V-JEPA 2-AC: Learning an Action-Conditioned World Model

[3.1 Action-Conditioned World Model Training](#)

[3.2 Inferring Actions by Planning](#)

4. Planning: Zero-shot Robot Control

[4.1 Experimental Setup](#)

[4.2 Results](#)

[4.3 Limitations](#)

5. Understanding: Probe-based Classification

6. Prediction: Probe-based Action Anticipation

7. Understanding: Video Question Answering

8. Related Work

9. Conclusion and Future Work

V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning

Background

Self-Supervised Learning(SSL) 세 가지 패러다임

- SSL: 사람이 붙인 라벨 없이 데이터 자체에서 학습 신호를 만들어내는 방식
- 무엇을 예측하느냐에 따라 크게 세 갈래로 나뉨

방식	예측 대상	대표 모델	특징
Generative (재구성)	가려진 픽셀 자체	MAE, VideoMAE	픽셀 단위 디테일까지 복원해야 함
Contrastive (대조)	같은/다른 쌍 구분	CLIP, SigLIP	텍스트 등 쌍 데이터 필요
JEPA (본 논문)	가려진 부분의 표현(embedding)	I-JEPA, V-JEPA	예측 가능한 추상 정보만 학습

- JEPA(Joint-Embedding Predictive Architecture): LeCun(2022)이 제안한 구조로, 픽셀이 아니라 **학습된 표현 공간에서** 예측
- 이렇게 하는 이유
 - 풀잎 하나하나의 위치, 나뭇잎의 흔들림처럼 본질적으로 예측 불가능한 디테일은 버림
 - 물체의 궤적처럼 예측 가능한 구조에 모델 용량을 집중시킴

ViT, 패치, Tubelet

- ViT(Vision Transformer): 이미지를 작은 패치로 잘라 각 패치를 토큰처럼 Transformer에 넣는 구조
- 이미지 패치: 16×16 픽셀 조각 하나가 토큰 1개
- 비디오 tubelet: 시간축까지 포함한 $2 \times 16 \times 16$ (T × H × W) 조각이 토큰 1개
→ 연속된 2프레임의 같은 위치를 한 덩어리로 묶는 셈

Representation Collapse와 EMA / Stop-gradient

- 문제: 표현 공간에서 "정답 표현"도 모델이 만들어내므로, 모든 입력에 같은 벡터를 내놓으면 loss가 0이 되는 **자명해(trivial solution)**가 존재
→ 이를 representation collapse라고 부름
- 해결책 두 가지
 - **Stop-gradient** $sg(\cdot)$: target 쪽으로는 gradient를 흘리지 않음
 - **EMA encoder**: target을 만드는 encoder의 가중치를 학습 중인 encoder 가중치의 지수이동평균으로 천천히 따라가게 함
$$\bar{\theta} \leftarrow m\bar{\theta} + (1 - m)\theta$$
 (논문은 $m = 0.99925$)
- 결과: target이 급변하지 않고 안정적으로 움직여 collapse 방지

Frozen Evaluation과 Attentive Probe

- Frozen evaluation: encoder 가중치는 얼리고, 그 위에 작은 분류기만 학습해서 표현의 품질을 평가
- Linear probe: 선형층 하나만 학습 (가장 엄격한 평가)
- **Attentive probe**: 작은 Transformer 블록 + learnable query token의 cross-attention으로 토큰들을 요약한 뒤 분류
→ 본 논문은 4-layer attentive probe 사용

World Model과 Model Predictive Control(MPC)

- World model: "현재 상태 + 행동 → 다음 상태"를 예측하는 내부 모델
- MPC: world model로 여러 행동 시퀀스를 머릿속에서 시뮬레이션해보고, 목표에 가장 가까워지는 행동을 골라 **첫 행동만 실행**한 뒤 다시 계획(receding horizon)
- CEM(Cross-Entropy Method): 가우시안에서 행동 후보를 샘플링
→ 성능 좋은 top-k로 평균/분산 갱신 → 반복하는 gradient-free 최적화

3D-RoPE

- RoPE(Rotary Position Embedding): 위치를 벡터에 더하는 대신 **회전**으로 인코딩해 상대 위치 정보를 표현
- 3D-RoPE: feature 차원을 3등분해 각각 시간 / 높이 / 너비 축에 1D RoPE를 적용
- 절대 위치 임베딩(sincos) 대비 대형 모델 학습 안정화에 도움

1. Introduction

1) 배경: 인간은 세계 모델을 가지고 있다

- 인지과학 이론들: 인간은 저수준 감각 입력을 통합해 세계의 내부 모델을 학습하고, 이를 통해 미래 상태를 표현/예측함
- 이 world model이 지각과 현실 이해를 형성하고, 행동의 결과를 예측해 목표 지향적 계획을 가능하게 함
- 목표 → 비디오 같은 감각 데이터로 world model을 학습해 **이해(understand)** → **예측(predict)** → **계획(plan)** 하는 에이전트 구축

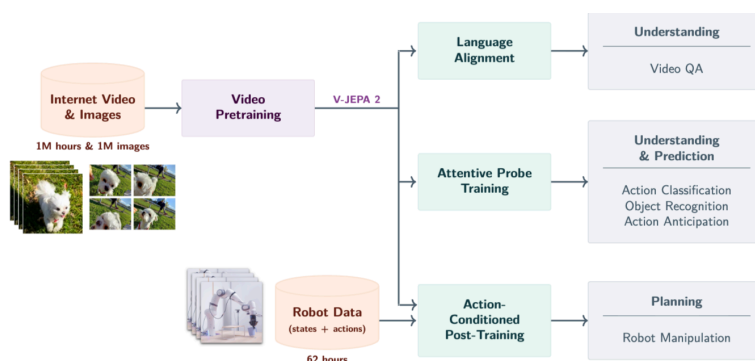
2) 기존 접근과 그 한계

- 상호작용 데이터(state-action 시퀀스) + 보상 기반 world model
→ 실세계 상호작용 데이터가 부족해 확장성이 제한됨
- 인터넷 비디오 + 상호작용 데이터로 **행동 조건부 비디오 생성 모델** 학습
→ 로봇 실행 결과는 제한적
→ 계획 능력보다 예측의 충실도/시각적 품질 평가에 치중 (비디오 생성으로 계획하는 연산 비용이 크기 때문으로 추정)

3) 본 논문의 아이디어

- JEPA로 표현 공간에서 예측하는 방식 채택
- vs 상호작용 데이터만 쓰는 방식 → 행동 정보 없는 인터넷 비디오도 활용 가능
- vs 비디오 생성 방식 → 예측 불가능한 픽셀 디테일은 무시하고 예측 가능한 측면에만 집중

4) 2단계 학습 구조



- Stage 1: 인터넷 비디오 100만 시간 + 이미지 100만 장으로 action-free 사전학습 (mask-denoising feature prediction)
- Stage 2: encoder를 열고, 로봇 데이터 62시간으로 action-conditioned predictor 학습 → V-JEPA 2-AC
- 사전학습된 V-JEPA 2는 세 방향으로 활용
 - Attentive probe → 행동 분류, 물체 인식, 행동 예측
 - LLM 정렬 → Video QA
 - Action-conditioned post-training → 로봇 조작 계획

5) 주요 기여 4가지

축	태스크	핵심 결과
Understanding (probe)	SSv2 동작 분류	top-1 77.3
Understanding (VidQA)	PerceptionTest 등	8B급 SOTA, 언어 지도 없는 encoder로 달성
Prediction	EK100 행동 예측	recall@5 39.7, 이전 최고 대비 상대 44% 향상
Planning	로봇 Grasp, Pick-and-Place	새 환경 zero-shot, 태스크별 학습/보상 없음

- "언어 지도 없이 사전학습된 비디오 encoder도 LLM과 정렬하면 SOTA 가능"
→ 기존 통념(언어 정렬된 encoder가 필요)과 반대되는 주장

2. V-JEPA 2: Scaling Self-Supervised Video Pretraining

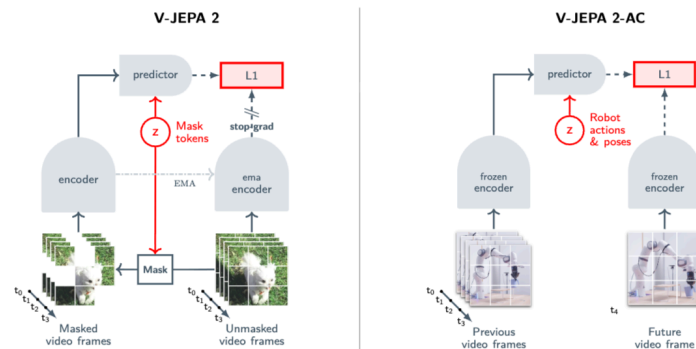
2.1 Methodology

1) Mask-Denoising in Representation Space (Eq. 1)

$$\text{minimize } \theta, \phi, \Delta_y; |P\phi(\Delta_y, E_\theta(x)) - sg(E_{\bar{\theta}}(y))|_1$$

기호	의미
y	원본 비디오
x	일부 패치를 무작위로 제거(mask)한 비디오
E_θ	encoder, 비디오 표현 추출
P_ϕ	predictor, 가려진 부분의 표현을 예측
Δ_y	learnable mask token, 가려진 패치의 위치를 알려줌
$E_{\bar{\theta}}$	EMA encoder, target 표현 생성
$sg(\cdot)$	stop-gradient

- 흐름



1. 비디오를 tubelet 토큰으로 쪼갬
2. 일부 토큰을 drop한 masked 시퀀스를 encoder에 입력
3. encoder 출력 + mask token을 concat해서 predictor에 입력
4. predictor 출력과 EMA encoder가 원본 비디오에서 뽑은 target 표현 사이 L1 loss 계산

- loss는 가려진 패치의 예측에만 적용
- encoder와 predictor를 동시에 학습

2) Architecture

- encoder, predictor 모두 ViT
- 절대 sincos 위치 임베딩 대신 3D-RoPE 사용 → 가장 큰 모델의 학습 안정화
- tubelet 크기 $2 \times 16 \times 16$, multiblock masking 전략(V-JEPA와 동일)
- predictor는 모든 encoder 크기에서 ViT-small급(22M)으로 고정

3) 네 가지 Key Scaling Ingredients

요소	V-JEPA (이전)	V-JEPA 2
데이터	2M 비디오	22M 비디오
모델	ViT-L (300M)	ViT-g (1B)
학습 길이	90K iter	252K iter
해상도	256px, 16프레임	384px, 64프레임

4) Evaluation Protocol

- 6개 태스크로 표현 품질 평가: SSv2, Diving-48, Jester (동작) / Kinetics, COIN, ImageNet (외형)
- encoder를 열고 태스크별 4-layer attentive probe 학습

2.2 Scaling Self-Supervised Video Learning

결과 (Figure 3): ViT-L/16 baseline에서 요소를 하나씩 추가하며 6개 태스크 평균 정확도 측정

단계	평균 정확도	증가분
V-JEPA 1 baseline	84.2	-
+ Data Scaling	85.2	+1.0
+ Model Scaling	86.7	+1.5
+ Long Training	87.5	+0.8
+ Higher Resolution	88.2	+0.7

- 누적 +4.0 → 모든 개별 요소가 양의 효과, 비디오 SSL의 스케일링 가능성 확인

2.3 Pretraining Dataset

1) VideoMix22M (VM22M) 구성 (Table 1)

소스	샘플 수	유형	큐레이션	샘플링 가중치
SSv2	168K	1인칭 비디오	X	0.056
Kinetics	733K	3인칭 비디오	X	0.188
HowTo100M	1.1M	3인칭 비디오	X	0.318
YT-Temporal-1B	19M	3인칭 비디오	O	0.188
ImageNet	1M	이미지	X	0.250

- 이미지는 16번 복제해 모든 프레임이 동일한 16프레임 비디오로 취급 → 이미지·비디오 공동 학습
- 공개 데이터만 사용해 재현 가능성 확보
- VM2M → VM22M: 평균 +1.0, 특히 Kinetics, COIN, ImageNet 같은 **외형 기반 태스크**에서 항상 폭이 큼 → 시각적 커버리지 확대 효과

2) Data Curation

- YT1B는 필터링이 거의 없어 노이즈(만화, 클립아트 등)가 많음
- 큐레이션 파이프라인 (Appendix A.2)
 1. 장면 전환 기준으로 클립 분할, 4초 미만 제거 → 3.16억 장면
 2. 각 장면 중간 프레임에서 DINOv2 임베딩 추출
 3. 150만 개 클러스터로 군집화
 4. target 데이터셋(Kinetics, SSv2, COIN, EpicKitchen)의 비디오를 가장 가까운 클러스터에 할당
 5. target이 하나라도 할당된 클러스터만 유지 → 약 21만 클러스터, 1.15억 장면
- 결과: 큐레이션된 YT1B가 미큐레이션 대비 평균 +1.4
- 단, ViT-L 규모에선 Curated YT1B 단독이 VM22M과 비슷했지만, 큰 모델일수록 VM22M 혼합의 이득이 커짐

2.4 Pretraining Recipe

1) Model Size Scaling

- ViT-L(300M) → ViT-H(600M) → ViT-g(1B)
- 동작(SSv2 +1.6)과 외형(Kinetics +1.5) 태스크 모두 향상 → 1B까지는 모델 용량을 효과적으로 활용

2) Training Schedule: Warmup-Constant-Decay

- 기존 cosine 스케줄 대신 warmup → constant → cooldown 구조
- 장점: constant 구간의 여러 체크포인트에서 cooldown을 따로 시작할 수 있어 긴 학습 탐색 비용이 절감됨
- EMA, weight decay 계수를 ramp-up 없이 고정 → 레시피 단순화 (성능 영향 미미)

3) Progressive-Resolution Training

- 문제: ViT-g를 64프레임 × 384px로 처음부터 학습하면 약 60 GPU-year 소요
- 해결
 - Warmup (12K iter): 16프레임, 256px
 - Constant (228K iter): 16프레임, 256px
 - Cooldown (12K iter): 프레임 수와 해상도를 올리면서 learning rate 선형 감소
- 비싼 고해상도·긴 클립 학습을 **마지막 cooldown에만** 몰아서 수행
- 결과: 풀해상도 학습 대비 GPU 시간 **8.4배 절감**

4) 시공간 해상도 스케일링

- 평가는 16프레임으로 고정해도, 사전학습 cooldown을 64프레임으로 하면 평균 +0.7
- 128, 256프레임까지 늘려봤지만 64프레임 이상에선 추가 향상 없음

3. V-JEPA 2-AC: Learning an Action-Conditioned World Model

이 섹션의 목적

- 사전학습된 V-JEPA 2는 비디오의 빈 부분을 예측할 수 있지만, 에이전트의 **행동이 미치는 인과적 효과**는 고려하지 않음
- 소량의 상호작용 데이터로 행동 조건부 predictor를 학습해 계획에 쓸 수 있게 만드는 단계 (Figure 2 오른쪽)

3.1 Action-Conditioned World Model Training

1) 데이터: Droid

- 원격조작으로 수집된 테이블 위 Franka Panda 7-DoF 로봇 팔 데이터
- "Unlabeled"의 의미 → 보상, 태스크 종류, 성공 여부 메타데이터를 쓰지 않음
- 원본 비디오 + end-effector 상태만 사용, **실패 궤적도 포함** (약 2.3만 궤적)
- 4초 미만 비디오를 버려서 62시간 미만만 사용

2) 모델 입력

- 4초 클립, 256px, 4fps → 16프레임 $(x_k)_{k \in [16]}$
- End-effector 상태 s_k : 7차원 벡터
 - 위치 3차원 + 방향(오일러 각) 3차원 + 그리퍼 상태 1차원
- 행동 a_k : 인접 프레임 간 end-effector 상태 변화량 (7차원), 총 15개

3) Loss Function

- 각 프레임을 **열린** V-JEPA 2 encoder로 독립적으로 인코딩 → $z_k \in \mathbb{R}^{16 \times 16 \times 1408}$
- (a_k, s_k, z_k) 를 시간순으로 interleave해서 predictor에 입력

Teacher-forcing loss (Eq. 2)

$$\mathcal{L}_{\text{teacher-forcing}}(\phi) = \frac{1}{T} \sum k = 1^T |P_\phi((a_t, s_t, E(x_t))t \leq k) - E(xk+1)|_1, \quad T = 15$$

- 매 시점마다 **실제** 현재 프레임 표현을 넣고 다음 프레임 표현을 예측

Rollout loss (Eq. 3)

$$\mathcal{L}_{\text{rollout}}(\phi) = |P\phi(a_{1:T}, s_1, z_1) - z_{T+1}|_1, \quad T = 2$$

- predictor의 **예측 출력을 다시 입력으로** 넣어 여러 스텝 앞을 예측
- 실제로는 recurrent step 1번만 미분

최종 목적함수 (Eq. 4): $\mathcal{L}(\phi) = \mathcal{L}_{\text{teacher-forcing}} + \mathcal{L}_{\text{rollout}}$

- Teacher forcing만 쓰면 추론 시 자기 예측을 입력으로 쓸 때 오차가 누적됨
→ rollout loss로 자기 출력에 대한 강건성을 학습시켜 **오차 누적 완화** (Figure 6)

4) Architecture

- 약 300M 파라미터 Transformer (24 layers, 16 heads, hidden 1024, GELU)
- 행동, 상태, feature map을 각각 별도의 affine 변환으로 hidden 차원에 매핑
- 위치 인코딩
 - 비디오 패치 → 3D-RoPE (시공간 위치)
 - 행동/자세 토큰 → 시간 축 RoPE만 적용
- **Block-causal attention**: 각 시점의 토큰은 같은 시점과 이전 시점의 토큰만 참조 가능 (미래 참조 불가)

3.2 Inferring Actions by Planning

1) Energy Minimization (Eq. 5)

$$\mathcal{E}(\hat{a}_{1:T}; z_k, s_k, z_g) = |P(\hat{a}_{1:T}; s_k, z_k) - z_g|_1$$

- z_k : 현재 관측 프레임의 표현
- z_g : 목표 이미지의 표현
- 의미 → "이 행동 시퀀스를 실행하면 T스텝 뒤 상상된 상태가 목표 상태와 얼마나 다른가"
- 이 energy를 최소화하는 행동 시퀀스 a^* 를 탐색

2) 최적화 방법 (Figure 7)

- CEM 사용
 1. 평균 0, 분산 1 가우시안에서 행동 후보 샘플링
 2. energy가 낮은 top-k 궤적의 통계로 평균/분산 갱신
 3. 여러 번 반복 후 최종 평균을 선택
- **첫 번째 행동만** 로봇에서 실행 → 새 관측 → 다시 계획 (receding horizon control)

4. Planning: Zero-shot Robot Control

4.1 Experimental Setup

1) Baselines

- **Octo**: Vision-Language-Action 모델, behavior cloning으로 학습
 - Open-X Embodiment(100만+ 궤적)로 사전학습된 가중치에서 시작
 - Droid 전체로 hindsight relabeling(궤적 후반 프레임을 목표로 재라벨링) fine-tuning
- **Cosmos**: 비디오 생성 기반 world model (latent diffusion 7B, 비디오 2천만 시간 학습)
 - 공식 행동 조건부 fine-tuning 코드로 Droid에 학습, 저자들이 성능 개선 위해 일부 설정 조정
 - 로봇 제어에 Cosmos를 사용한 첫 시도라고 주장

2) 배포 환경

- 서로 다른 두 연구실의 Franka 팔, 둘 다 **Droid에 없는 환경**
- 보정되지 않은 저해상도 단안 RGB 카메라
- 각 행동은 반경 0.075의 L1-ball로 제한 → 한 번에 최대 약 13cm 이동 (큰 행동은 분포 밖이라서)

4.2 Results

1) Single-Goal Reaching (Figure 8)

- 목표 이미지 하나만 보고 end-effector를 해당 위치로 이동
- 모든 경우 목표 4cm 이내 도달, 오차가 **단조 감소**
- 일종의 visual servoing이지만, 라벨 없는 실세계 비디오만으로 달성했다는 점이 차별점

2) Energy Landscape (Figure 9)

- Δx , Δy 를 바꿔가며 energy 시각화
- 최솟값이 정답 행동 근처에 위치 (정답 (0, -0.1), 최솟값 약 (0, -0.05))
- landscape가 비교적 **매끄럽고 국소적으로 볼록** → 계획 최적화가 쉬운 구조

3) Prehensile Manipulation (Table 2)

태스크	Octo (평균)	V-JEPA 2-AC (평균)
Reach	100%	100%
Grasp - Cup	15%	65%
Grasp - Box	0%	25%
Reach w/ Obj - Cup	15%	75%
Reach w/ Obj - Box	70%	75%
Pick-&-Place - Cup	15%	80%
Pick-&-Place - Box	10%	65%

- 태스크당 10회 시행 평균
- Pick-and-Place는 목표 이미지 외에 **서브골 이미지 2장**을 추가로 제공
 - 서브골1(물체 잡는 모습) 4스텝 → 서브골2(목표 근처) 10스텝 → 최종 목표 4스텝
- 물체별 난이도 차이
 - 컵: 손가락 하나를 안쪽에 넣고 테두리를 잡는 방식, 행동이 부정확하면 테두리를 놓침
 - 박스: 잡는 방법은 많지만 그리퍼를 충분히 벌리는 정밀 제어 필요

4) Cosmos와의 계획 비교 (Table 3, Lab 2)

	Cosmos	V-JEPA 2-AC
샘플 수	80	800
행동 1개 계산 시간	4분	16초
Reach	80%	100%
Grasp Cup / Box	0% / 20%	60% / 20%
Pick-&-Place Cup / Box	0% / 0%	80% / 50%

- 샘플을 10배 더 쓰고도 약 15배 빠름 → 픽셀 생성 없이 표현 공간에서 예측하는 효율성
- Cosmos로는 Pick-and-Place 한 번에 1시간 이상 소요

4.3 Limitations

1) 카메라 위치 민감성

- 카메라 보정 없이 행동 좌표축을 영상에서 암묵적으로 추론해야 함
- 로봇 base가 화면에 안 보이면 좌표축 추론이 불명확 → world model 오차
- 실제로 여러 카메라 위치를 수동으로 시도한 후 잘 되는 위치로 고정
- Appendix B.4: 추론된 좌표축의 회전 오차가 카메라 각도에 거의 **선형 비례**

2) Long-horizon 계획

- Autoregressive 예측의 오차 누적
- 계획 horizon이 선형으로 늘면 탐색 공간은 지수적으로 증가
- 실험은 사실상 planning horizon 1 → Pick-and-Place도 서브골 없이는 어려움

3) 이미지 목표

- 현재는 목표를 이미지로 지정해야 함 → 언어 목표로의 확장이 향후 과제

5. Understanding: Probe-based Classification

1) 이 섹션의 의미

- V-JEPA 2-AC 같은 표현 공간 world model의 능력은 **표현이 담고 있는 정보**에 의해 원천적으로 제한됨
- 따라서 표현 자체의 품질을 분류 태스크로 검증

2) 태스크 구성

- 동작 이해 (여러 프레임 필요): SSv2, Diving-48, Jester
- 외형 이해 (한 프레임으로도 해결 가능): Kinetics400, COIN, ImageNet

3) 결과 (Table 4)

모델	평균	SSv2	K400	IN1K
DINOv2 (이미지 SSL)	81.1	50.7	83.6	86.1
PEcoreG (이미지-텍스트)	82.3	55.4	88.5	87.6
SigLIP2 (이미지-텍스트)	81.1	49.9	87.3	88.0
V-JEPA ViT-H	85.2	74.3	84.5	80.0
InternVideo2s2-1B	87.0	69.7	89.4	85.8
V-JEPA 2 ViT-g	87.5	75.3	86.6	84.6
V-JEPA 2 ViT-g384	88.2	77.3	87.3	85.1

- 동작 이해에서 압도적 (SSv2 75.3 vs 이미지-텍스트 모델 55 전후)
- 외형 이해에선 경쟁력 있는 수준이지만 **최고는 아님** (ImageNet, K400은 언어 지도 모델이 우세)
- 주의: baseline들은 학습 데이터가 서로 달라 **시스템 수준 비교**만 가능하다고 저자들이 명시

6. Prediction: Probe-based Action Anticipation

1) 태스크: Epic-Kitchens-100 (EK100)

- 45개 주방에서 1인칭으로 촬영한 요리 영상 100시간
- 행동 라벨 3,568개 = 동사(97) × 명사(300) 조합
- 행동 시작 **1초 전**까지의 영상을 보고 다음 행동(동사, 명사, 행동)을 예측
- 가능한 미래가 여러 개이므로 mean-class recall@5로 평가

2) Anticipation Probe

1. 행동 시작 1초 전에 끝나는 클립을 encoder에 입력

2. predictor가 encoder 출력 + 1초 뒤 프레임 위치의 mask token으로 미래 프레임 표현 예측
3. encoder 출력과 predictor 출력을 concat해 attentive probe에 입력
4. query token 3개 → 동사/명사/행동 분류기 각각, focal loss 합산

3) 결과 (Table 5)

모델	파라미터	Action recall@5
InAViT	160M	25.8
Video-LLaMA	7B	26.0
PlausiVL	8B	27.6
V-JEPA 2 ViT-L	300M	32.7
V-JEPA 2 ViT-H	600M	36.5
V-JEPA 2 ViT-g	1B	38.0
V-JEPA 2 ViT-g384	1B	39.7

- 모델 크기에 따라 거의 선형 향상
- 300M 모델이 8B PlausiVL을 이미 능가, 최고 모델은 +12.1 (상대 44% 향상)

4) 정성 분석 (Figure 11)

- 성공 사례: 정답을 top-1로 맞히고 top 2-5도 맥락상 그럴듯한 행동 제시
- 실패 사례: "close door" 같은 그럴듯한 행동은 내놓지만 물체 종류(tea package)를 틀림

5) Limitations

- EK100을 완전히 해결하진 못함
- 예측 시간이 1초보다 길어지면 성능 급락
- 주방 환경, 닫힌 어휘에 한정 → 다른 환경 일반화 미지수

7. Understanding: Video Question Answering

1) 설정

- LLaVA 방식: vision encoder 출력 → projector(MLP 등) → LLM 입력 임베딩 공간
- 기존 MLLM은 대부분 CLIP, SigLIP 같은 언어 정렬된 이미지 encoder를 프레임별로 적용
- 본 논문 → 언어 지도 없이 사전학습된 비디오 encoder로 MLLM을 학습한 첫 사례라고 주장
- 3단계 학습: 이미지 캡셔닝으로 projector 정렬 → 대규모 이미지 QA → 대규모 비디오 캡셔닝/QA

2) 이미지 encoder와의 통제 비교 (Table 6)

- 동일 조건: Qwen2-7B-Instruct, 18M 샘플, encoder frozen

Encoder	평균	PerceptionTest	MVP	TemporalBench	TVBench
DINOv2	45.7	67.1	22.4	26.8	47.6
SigLIP2	48.1	72.4	26.2	25.7	48.7
PE	49.1	72.3	26.7	27.5	51.6
V-JEPA 2	52.3	72.0	31.1	33.3	55.9

- PerceptionTest만 근소하게 뒤지고 나머지는 모두 우위
- 특히 시간적 이해 벤치마크(MVP, TemporalBench, TVBench)에서 격차가 큼

3) Encoder 크기 및 해상도 스케일링 (Table 7)

- End-to-end 학습(encoder 학습 허용) 기준

- ViT-L256 51.7 → ViT-g256 52.3 → ViT-g384 54.0 → ViT-g512 54.4
- encoder와 해상도를 키울수록 일관된 향상

4) 데이터 스케일링으로 SOTA (Table 8)

- 정렬 데이터 18M → 88.5M, LLM은 Llama 3.1 8B, ViT-g384 (프레임당 288 토큰)

모델	평균	PerceptionTest	MVP	TempCompass	TOMATO	MVBench
Qwen2.5VL	49.7	70.5	36.7	71.7	24.6	69.6
PLM 8B	56.7	82.7	39.7	72.7	33.2	77.1
V-JEPA 2	59.5	84.0	44.5	76.9	40.3	73.5

- 5개 벤치마크 SOTA, TVBench와 MVBench에선 PLM에 뒤짐
- 단, PerceptionTest는 V-JEPA 2만 SFT 후 test set 결과이고 다른 모델은 zero-shot (Table 8 캡션에 명시)

8. Related Work

1) World Models and Planning

- 픽셀 공간 / 학습된 표현 공간 / 키포인트 같은 구조적 공간에서 예측하는 연구들 존재
- 기존 실세계 로봇 연구 → 태스크 특화 world model, 배포 환경의 상호작용 데이터에 의존
- 본 논문 → 태스크 무관(task-agnostic) world model로 새 환경/물체 일반화 시연
- Genie, Cosmos 등 비디오 생성 world model → 그럴듯한 영상 생성은 보였지만 실제 로봇 제어는 입증 못 함

2) VLA 모델

- RT-2, OpenVLA, $\pi 0$ 등: 인터넷 사전학습 비디오-언어 모델을 expert 시연으로 behavior cloning
- 한계 → 명시적 world model이 없어 추론 시 계획 불가, 고품질 원격조작 데이터와 **성공 궤적만** 활용 가능
- 본 논문 → 성공/실패 상관없이 모든 상호작용 데이터 활용

3) Vision Foundation Models

- 이미지/비디오 SSL, 약한 언어 지도 등으로 범용 vision encoder 학습
- 기존 연구는 probe 평가나 VQA 같은 이해 평가에 치중
- 본 논문 → 이해를 넘어 물리 세계와의 상호작용(계획)까지 확장

9. Conclusion and Future Work

1) 결론

- JEPA를 웹 규모 데이터 + 소량 로봇 데이터로 학습해 이해, 예측, 계획이 가능한 world model 구축
- 동작 이해 분류, 행동 예측에서 SOTA
- LLM 정렬 시 VidQA에서 기존 vision encoder 능가
- V-JEPA 2-AC로 실제 로봇에서 zero-shot Pick-and-Place 성공

2) 향후 연구 방향

1. 약 16초 이상의 장기 예측 → 여러 시공간 스케일과 추상화 수준에서 예측하는 **계층적 모델**
2. 이미지 목표 대신 **언어 목표**를 V-JEPA 2-AC 표현 공간에 임베딩
3. 1B를 넘어 20B급까지 스케일링 가능한 사전학습 레시피 개발